LAPORAN EVALUASI TENGAH SEMESTER (ETS)

ETS SRO

# Pemetaan 3D Otonom dengan Drone Quadcopter pada Simulator CoppeliaSim


Nama : Keindra Dante Hidayat
NRP : 5022231236
Dosen : Muh. Qomaruz Zaman, S.T., M.T., Ph.D.

Teknik Elektro / Institut Teknologi Sepuluh Nopember


**Tautan Proyek**

**Repositori GitHub:**
https://github.com/dante011111/ETS-SRO-Drone-Mapping

**Video Demonstrasi 1 (Google Drive):**
https://drive.google.com/file/d/11Uf6JHnGKmAgEEihSATajqX0j-cRtIVX/view?usp=drive_link

**Video Demonstrasi 2 (Google Drive):**
https://drive.google.com/file/d/1lNExSDm8HoHWYaXFC_Jeajzcxdrl0UzW/view?usp=drive_link

28 Oktober 2026

# Daftar Isi

**1 Pendahuluan** **3**

1.1 Latar Belakang . . . 3

1.2 Tujuan . . . 3

1.3 Ruang Lingkup . . . 3

**2 Deskripsi Tugas** **3**

**3 Landasan Teori** **4**

3.1 Point Cloud . . . 4

3.2 Model Kamera Kedalaman (Pinhole) . . . 4

3.3 Voxel Downsampling . . . 4

3.4 Pola Lintasan Lawn-Mower . . . 5

**4 Perangkat dan Lingkungan Simulasi** **5**

4.1 Perangkat Lunak . . . 5

4.2 Scene dan Objek yang Dipetakan . . . 5

4.3 Konfigurasi Sensor . . . 6

4.4 Parameter Utama Skrip . . . 6

**5 Arsitektur Sistem** **7**

5.1 Struktur Repositori . . . 7

5.2 Komunikasi dengan Simulator . . . 7

**6 Cara Kerja Sistem** **7**

6.1 Alur Program Utama . . . 7

6.2 Kontrol Drone Otonom . . . 8

6.3 Rute Transit . . . 8

6.4 Misi Path (Grade C) . . . 9

6.5 Pemindaian 2D (Grade BC) . . . 9

6.6 Pemindaian 3D (Grade B) . . . 9

6.7 Akuisisi Data Sensor . . . 10

**7 Pengolahan Data (Post-Processing)** **11**

7.1 Pembersihan Data . . . . . 11
7.2 Voxel Downsampling (Grade AB) . . . . . 11
7.3 Pewarnaan Berdasarkan Ketinggian (Grade A) . . . . . 11
7.4 Visualisasi dan Keluaran . . . . . 11

**8 Cara Menjalankan** **12**
8.1 Persiapan . . . . . 12
8.2 Pengambilan Data . . . . . 12
8.3 Pengolahan Data . . . . . 12
8.4 Pemecahan Masalah . . . . . 13

**9 Hasil dan Pembahasan** **13**
9.1 Hasil Visualisasi . . . . . 13
9.2 Ringkasan Angka Hasil . . . . . 14
9.3 Pembahasan . . . . . 14

**10 Kesimpulan** **15**

**Lampiran: Tautan Proyek** **15**

# 1 Pendahuluan

## 1.1 Latar Belakang

Pemetaan lingkungan merupakan kemampuan dasar bagi robot bergerak, termasuk pesawat tanpa awak (*drone*). Dengan membawa sensor jarak atau kamera kedalaman, sebuah drone dapat terbang di atas suatu area, merekam jarak ke permukaan di sekitarnya, lalu menyusun hasil rekaman tersebut menjadi model tiga dimensi berbentuk *point cloud*. Model seperti ini dipakai pada inspeksi infrastruktur, survei lahan, dan pemantauan area yang sulit dijangkau.

Proyek ini mensimulasikan proses tersebut di **CoppeliaSim**. Sebuah *quadcopter* dikendalikan secara otonom melalui skrip Python untuk terbang di atas area yang berisi lima objek geometris (kotak dan silinder) dengan tinggi berbeda-beda. Selama terbang, drone memindai lingkungannya, kemudian data yang terkumpul diolah menjadi peta 2D dan peta 3D berwarna.

## 1.2 Tujuan

Tujuan proyek ini adalah:

1. mengendalikan drone secara otonom melalui lintasan waypoint dan memvisualisasikan jalur terbangnya (Grade C);
2. membuat pemindaian 2D menggunakan sensor kedalaman satu baris yang berputar (Grade BC);
3. membuat pemindaian 3D menggunakan kamera kedalaman yang dimiringkan dengan pola lintasan *lawn-mower* (Grade B);
4. mengurangi jumlah titik pada *point cloud* melalui *voxel downsampling* (Grade AB);
5. mewarnai peta berdasarkan ketinggian (sumbu Z) (Grade A).

## 1.3 Ruang Lingkup

Sistem berjalan sepenuhnya di simulasi. Posisi drone diambil langsung dari simulator (*ground truth*), sehingga tidak ada proses estimasi posisi atau SLAM. Sensor kedalaman pada simulasi juga tidak memiliki derau (*noise*). Batasan ini dibahas lebih lanjut pada Bagian 9.

# 2 Deskripsi Tugas

Tugas dinilai secara bertingkat, dan setiap tingkat mencakup pekerjaan tingkat sebelumnya. Tabel 1 merangkum persyaratan tiap grade beserta cara repositori ini memenuhinya.

Tabel 1: Persyaratan tugas dan implementasinya

| **Grade** | **Persyaratan** | **Implementasi** |
|---|---|---|
| C | Kontrol drone otonom dan visualisasi jalur terbang | `drone_mapping.py --mode path`; jalur digambar sebagai garis merah di scene |
| BC | Pemindaian 2D dengan sensor kedalaman satu baris yang berputar | `drone_mapping.py --mode 2d`; hasil pada `scan2d.npy` |
| B | Pemindaian 3D dengan kamera kedalaman miring dan lintasan *lawn-mower* | `drone_mapping.py --mode 3d`; hasil pada `scan3d.npy` |
| AB | Optimasi *point cloud* dengan *voxel downsampling* | `postprocess.py --voxel 0.15` |
| A | Pewarnaan peta berdasarkan ketinggian | `postprocess.py`; *colormap* `RdYlGn_r` pada sumbu Z |

# 3 Landasan Teori

## 3.1 Point Cloud

*Point cloud* adalah kumpulan titik $(x, y, z)$ dalam satu kerangka koordinat. Setiap titik mewakili satu lokasi pada permukaan objek yang terkena pengukuran sensor. Kepadatan titik menentukan detail peta, tetapi juga menentukan beban penyimpanan dan komputasi.

## 3.2 Model Kamera Kedalaman (Pinhole)

Kamera kedalaman menghasilkan citra berukuran $N \times N$ piksel, dengan setiap piksel menyimpan jarak sepanjang sumbu optik, $z$. Untuk sudut pandang (*field of view*) $\phi$, panjang fokus dalam satuan piksel adalah

$$f = \frac{N/2}{\tan(\phi/2)}. \tag{1}$$

Piksel pada kolom $u$ dan baris $v$ dengan kedalaman $z$ berkorespondensi dengan titik pada kerangka sensor

$$x_s = -\frac{u - \frac{N-1}{2}}{f}\, z, \qquad y_s = \frac{v - \frac{N-1}{2}}{f}\, z, \qquad z_s = z. \tag{2}$$

Dengan $N = 128$ dan $\phi = 60°$, diperoleh $f \approx 110{,}9$ piksel.

## 3.3 Voxel Downsampling

Ruang dibagi menjadi kubus kecil (*voxel*) berukuran sisi $s$. Semua titik yang jatuh dalam satu voxel diganti dengan satu titik wakil. Indeks voxel dari titik $\mathbf{p}$ dan pusat voxel-nya

adalah

$$\mathbf{i} = \left\lfloor \frac{\mathbf{p}}{s} \right\rfloor, \qquad \mathbf{c} = (\mathbf{i} + 0{,}5)\, s. \tag{3}$$

Hasilnya adalah *point cloud* yang lebih ringan dan lebih seragam kepadatannya.

### 3.4 Pola Lintasan Lawn-Mower

Pola *lawn-mower* adalah lintasan bolak-balik paralel seperti orang memotong rumput. Pola ini menjamin seluruh area dilewati dengan jarak antar jalur yang tetap, sehingga cakupan pemindaian merata.

## 4 Perangkat dan Lingkungan Simulasi

### 4.1 Perangkat Lunak

- **CoppeliaSim** versi 4.5 atau lebih baru (dukungan ZMQ Remote API).
- **Python 3.x** dengan paket `coppeliasim-zmqremoteapi-client`, `numpy`, dan `matplotlib`.
- `open3d` (opsional) untuk tampilan 3D interaktif.

### 4.2 Scene dan Objek yang Dipetakan

File scene utama adalah `ETS Mapping.ttt`. Scene berisi lantai pada $z = 0$, model *Quadcopter* beserta *dummy* bernama `target`, dan lima objek statis. Jika objek `obstacle_0` belum ada di scene, skrip akan membuatnya secara otomatis lewat fungsi `build_scene`. Tabel 2 menunjukkan ukuran dan posisi objek.

Tabel 2: Objek yang dipetakan (ukuran $x \times y \times z$ dalam meter)

| **No.** | **Bentuk** | **Ukuran (m)** | **Posisi** $(x, y)$ | **Tinggi puncak (m)** |
|---|---|---|---|---|
| 0 | Kotak | $1{,}0 \times 1{,}0 \times 1{,}5$ | $(-2{,}0;\ -2{,}0)$ | 1,5 |
| 1 | Silinder | $1{,}0 \times 1{,}0 \times 2{,}0$ | $(2{,}0;\ -1{,}5)$ | 2,0 |
| 2 | Kotak | $1{,}5 \times 0{,}8 \times 1{,}2$ | $(0{,}0;\ 1{,}5)$ | 1,2 |
| 3 | Silinder | $0{,}8 \times 0{,}8 \times 1{,}4$ | $(-2{,}5;\ 2{,}5)$ | 1,4 |
| 4 | Kotak | $1{,}0 \times 1{,}0 \times 2{,}2$ | $(2{,}5;\ 2{,}5)$ | 2,2 |

Setiap objek diberi warna berbeda dan ditetapkan sebagai objek statis agar tidak bergeser selama simulasi.

## 4.3 Konfigurasi Sensor

Sensor yang dipakai adalah *Perspective Vision Sensor* bernama `mapping_sensor`, yang dibuat manual di CoppeliaSim dan dijadikan *child* dari `Quadcopter`. Pengaturannya tercantum pada Tabel 3.

Tabel 3: Konfigurasi `mapping_sensor`

| **Parameter** | **Nilai** |
|---|---|
| Resolusi X / Y | 128/128 piksel |
| Near clipping plane | 0,50 m |
| Far clipping plane | 12,00 m |
| Perspective angle | 60° |
| Explicit handling | Aktif (dicentang) |

Opsi *Explicit handling* wajib diaktifkan agar sensor hanya mengambil gambar ketika dipanggil oleh skrip Python (`sim.handleVisionSensor`), bukan setiap langkah simulasi. Nilai *near plane* 0,5 m sengaja dipilih agar badan drone sendiri tidak ikut terekam.

## 4.4 Parameter Utama Skrip

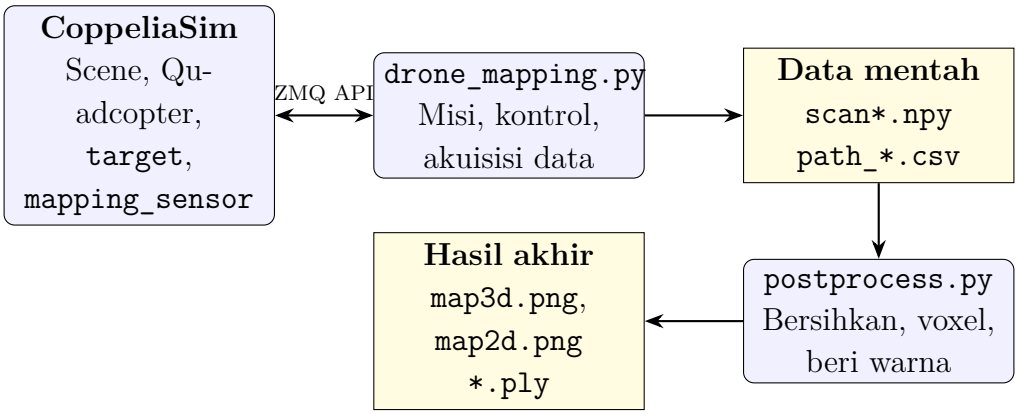
Seluruh parameter berada di bagian atas `drone_mapping.py` dan dapat disesuaikan (Tabel 4).

Tabel 4: Parameter utama pada `drone_mapping.py`

| **Parameter** | **Nilai** | **Keterangan** |
|---|---|---|
| `RES` | 128 | Resolusi sensor (persegi) |
| `FOV` | 60° | Sudut pandang |
| `NEAR`, `FAR` | 0,5; 12,0 m | Bidang potong sensor |
| `SPEED` | 1,5 m/s | Kecepatan gerak *target* |
| `AREA` | 4,0 m | Setengah sisi area terbang |
| `Z_SAFE` | 3,5 m | Ketinggian transit |
| `Z_PATH` | 2,5 m | Ketinggian misi *path* |
| `Z_2D` | 1,0 m | Ketinggian pemindaian 2D |
| `Z_3D` | 3,0 m | Ketinggian pemindaian 3D |
| `PITCH_3D` | 45° | Kemiringan sensor ke bawah |
| `SCAN_DT_3D` | 0,2 s | Selang waktu antar pemindaian 3D |
| `PIXEL_STRIDE_3D` | 2 | Ambil setiap piksel ke-2 (4096 titik/frame) |
| `SPIN_RATE` | 180°/s | Kecepatan putar sensor pada mode 2D |

# 5 Arsitektur Sistem

Sistem terdiri atas dua bagian yang terpisah: (1) pengambilan data yang berinteraksi langsung dengan simulator, dan (2) pengolahan data yang berjalan secara *offline*. Pemisahan ini membuat pengolahan dapat diulang dengan parameter berbeda (misalnya ukuran voxel) tanpa menjalankan ulang penerbangan.




Gambar 1: Diagram alur sistem

## 5.1 Struktur Repositori

```
.
|-- ETS Mapping.ttt # Scene utama CoppeliaSim
|-- drone_mapping.py # Kontrol drone dan pengambilan data
|-- postprocess.py # Pembersihan, voxel, pewarnaan, plot
|-- readme.md # Dokumentasi proyek
‘-- output/ # Dibuat otomatis: CSV, NPY, PNG, PLY
```

## 5.2 Komunikasi dengan Simulator

Skrip Python berkomunikasi dengan CoppeliaSim melalui **ZMQ Remote API** (`RemoteAPIClient`). Skrip menyalakan mode *stepping* (`client.setStepping(True)`) sehingga simulasi hanya maju satu langkah setiap kali skrip memanggil `client.step()`. Akibatnya, gerak drone, pembacaan posisi, dan pengambilan gambar sensor selalu sinkron dan hasilnya dapat diulang.

# 6 Cara Kerja Sistem

## 6.1 Alur Program Utama

Fungsi `main` menjalankan urutan berikut.

1. Membaca argumen `--mode` (`path`, `2d`, atau `3d`) dan opsi `--manual`.
2. Terhubung ke CoppeliaSim dan menghentikan simulasi yang masih berjalan.
3. Mencari objek `Quadcopter`, *dummy* `target`, dan `mapping_sensor`; membuat lima objek uji bila belum ada.
4. Menyalakan mode *stepping*, memulai simulasi, dan mencatat posisi awal `target` sebagai titik *start*.
5. Membentuk daftar waypoint sesuai mode, lalu menerbangkan drone (`fly`) sambil mencatat jalur dan mengambil data sensor.
6. Menyimpan `path_<mode>.csv` (kolom `t,x,y,z`) dan, untuk mode 2D/3D, `scan<mode>.npy`.
7. Menghentikan simulasi dan mencetak `[done]`.

Skrip harus dibiarkan sampai mencetak `[done]`, karena data baru disimpan setelah seluruh lintasan selesai.

## 6.2 Kontrol Drone Otonom

Model *Quadcopter* bawaan CoppeliaSim memiliki skrip internal yang membuat drone mengikuti posisi sebuah *dummy* bernama `target`. Skrip Python tidak mengatur rotor secara langsung. Skrip hanya menggeser `target`, dan drone mengejarnya.

Pada setiap langkah simulasi, `target` digeser menuju waypoint tujuan $\mathbf{w}$ sejauh $v\,\Delta t$:

$$\mathbf{t}_{k+1} = \mathbf{t}_k + (\mathbf{w} - \mathbf{t}_k)\min\!\left(1,\ \frac{v\,\Delta t}{\|\mathbf{w} - \mathbf{t}_k\|}\right), \tag{4}$$

dengan $v = \texttt{SPEED} = 1{,}5$ m/s dan $\Delta t$ adalah langkah waktu simulasi. Gerak ini berulang hingga `target` tiba di waypoint, lalu berlanjut ke waypoint berikutnya. Arah hadap (*yaw*) kamera ditetapkan searah gerak horizontal:

$$\psi = \operatorname{atan2}\left(w_y - t_y,\ w_x - t_x\right). \tag{5}$$

Setelah semua waypoint selesai, skrip menunggu 3 detik agar drone stabil.

Setiap langkah, posisi drone dibaca dan ditambahkan ke log jalur, dan sebuah ruas garis merah digambar di scene dengan `sim.addDrawingObject`. Garis ini adalah visualisasi jalur terbang untuk Grade C.

## 6.3 Rute Transit

Fungsi `transit_wrap` membungkus setiap rute pemindaian agar drone tidak menabrak objek:

1. naik ke ketinggian aman $Z_{\text{safe}} = 3{,}5$ m di atas titik awal;
2. terbang mendatar ke titik pertama rute;

3. turun ke ketinggian misi;
4. terbang sepanjang rute (di sinilah pemindaian aktif);
5. naik kembali ke $Z_{\text{safe}}$ dan kembali ke titik awal.

Ketinggian aman 3,5 m berada di atas objek tertinggi (2,2 m).

## 6.4 Misi Path (Grade C)

Drone terbang mengelilingi persegi berukuran $8 \times 8$ m (dari $-4$ sampai 4 pada sumbu $x$ dan $y$) pada ketinggian 2,5 m. Tidak ada pemindaian pada mode ini, hanya pencatatan dan penggambaran jalur. Tersedia juga opsi `--manual N`: skrip hanya merekam dan menggambar jalur selama $N$ detik sementara pengguna menggeser `target` sendiri lewat antarmuka CoppeliaSim.

## 6.5 Pemindaian 2D (Grade BC)

Mode 2D meniru *lidar* dua dimensi menggunakan satu baris piksel pada sensor kedalaman.

- Drone terbang pada ketinggian 1,0 m mengikuti rute persegi yang sama. Semua objek lebih tinggi dari 1,0 m, sehingga semuanya terpotong oleh irisan pemindaian.
- Hanya baris tengah citra (`RES//2`) yang dipakai, dengan 128 kolom.
- Sensor tidak dimiringkan ($pitch = 0$) dan diputar terhadap sumbu vertikal dengan sudut $\psi(t) = \texttt{SPIN_RATE} \cdot t$, yaitu 180°/s, sehingga satu putaran penuh memerlukan 2 detik.
- Pemindaian dilakukan pada setiap langkah simulasi.

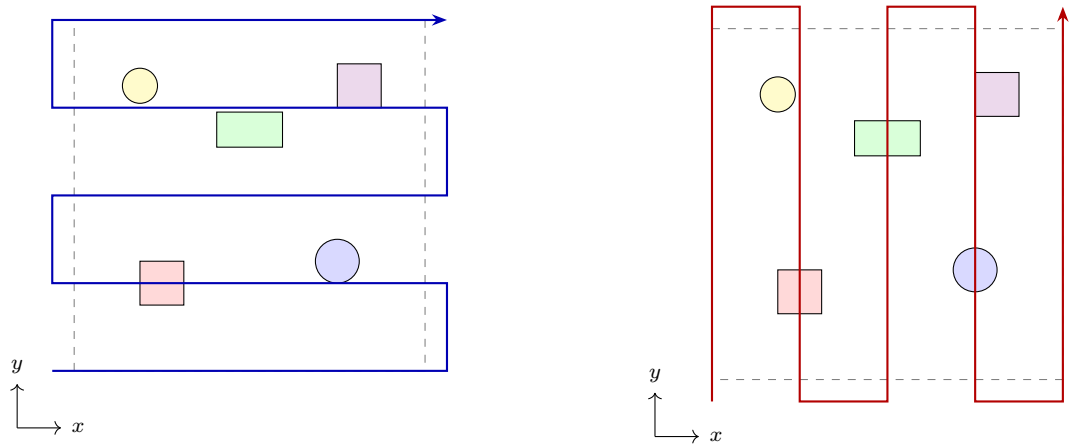
Hasilnya adalah titik-titik tabrakan pada satu bidang horizontal yang dapat ditampilkan sebagai peta tampak atas.

## 6.6 Pemindaian 3D (Grade B)

Mode 3D memakai seluruh citra kedalaman dan menggerakkan drone dengan pola *lawn-mower*.

- Ketinggian terbang 3,0 m. Sensor dimiringkan 45° ke bawah sehingga dalam satu citra terlihat tanah, dinding objek, dan bagian atas objek.
- Sensor menghadap searah gerak ($\psi$ dari arah waypoint).
- Pemindaian dilakukan tiap 0,2 detik. Hanya setiap piksel ke-2 yang dipakai, sehingga tiap frame menghasilkan paling banyak $64 \times 64 = 4096$ titik.
- Lintasan dilakukan dalam **dua tahap**: lajur sejajar sumbu $X$, kemudian lajur sejajar sumbu $Y$, dengan jarak antar lajur 2 m (lajur pada $-4, -2, 0, 2, 4$) dan

perpanjangan 0,5 m di kedua ujung. Sebuah titik dilihat dari dua arah yang berbeda sehingga sisi objek yang tertutup pada satu arah tetap terekam pada arah lainnya.




(a) Tahap 1: lajur sejajar sumbu $X$

(b) Tahap 2: lajur sejajar sumbu $Y$

Gambar 2: Pola lintasan *lawn-mower* pada mode 3D (tampak atas) beserta posisi kelima objek

## 6.7 Akuisisi Data Sensor

Pengambilan satu frame dilakukan oleh fungsi `acquire`, dengan langkah:

1. **Menyusun pose sensor.** Sensor diletakkan pada posisi drone $\mathbf{p}$, menghadap arah $\psi$, dan dimiringkan ke bawah sebesar $\theta$. Sumbu sensor dihitung sebagai

$$\mathbf{z}_s = \begin{bmatrix} \cos\psi\cos\theta & \sin\psi\cos\theta & -\sin\theta \end{bmatrix}^\top, \tag{6}$$

$$\mathbf{y}_s = \begin{bmatrix} \sin\theta\cos\psi & \sin\theta\sin\psi & \cos\theta \end{bmatrix}^\top, \tag{7}$$

$$\mathbf{x}_s = \mathbf{y}_s \times \mathbf{z}_s, \tag{8}$$

   dengan $+\mathbf{z}_s$ sebagai arah pandang dan $+\mathbf{y}_s$ sebagai arah atas citra. Matriks transformasi $3 \times 4$ $\mathbf{M} = [\mathbf{R} \,|\, \mathbf{p}]$ dengan $\mathbf{R} = [\mathbf{x}_s \; \mathbf{y}_s \; \mathbf{z}_s]$ lalu dipasang pada sensor melalui `setObjectMatrix` terhadap kerangka dunia. Dengan cara ini orientasi sensor tidak terpengaruh oleh kemiringan badan drone saat bermanuver.

2. **Mengambil gambar.** Skrip memanggil `sim.handleVisionSensor` lalu `sim.getVisionSensorDe 0)`. Argumen `0` membuat kedalaman dikembalikan dalam nilai ternormalisasi $d \in [0, 1]$ antara *near* dan *far plane*.

3. **Mengubah ke jarak metrik.** Kedalaman metrik adalah

$$z = \texttt{NEAR} + d\,(\texttt{FAR} - \texttt{NEAR}). \tag{9}$$

   Piksel dengan $d \geq 0{,}995$ dianggap tidak mengenai apa pun (*no hit*, jatuh di *far plane*) dan dibuang.

4. **Proyeksi balik ke kerangka sensor** memakai Persamaan (1) dan (2).

5. **Transformasi ke kerangka dunia:**

$$\mathbf{p}_w = \mathbf{R}\,\mathbf{p}_s + \mathbf{p}. \tag{10}$$

Titik-titik dari seluruh frame ditumpuk (`np.vstack`) dan disimpan sebagai larik `float32` berukuran $N \times 3$ pada `scan2d.npy` atau `scan3d.npy`.

Jika peta tampak terbalik (*mirrored*), variabel `FLIP_U` atau `FLIP_V` pada bagian atas skrip dapat diubah menjadi `True`, lalu pemindaian diulang.

# 7 Pengolahan Data (Post-Processing)

Skrip `postprocess.py` membaca data mentah dari folder `output` (atau folder kerja saat ini), kemudian melakukan tiga tahap berikut.

## 7.1 Pembersihan Data

Fungsi `clean` membuang titik yang bernilai `NaN` atau tak hingga dan titik yang berada jelas di bawah tanah ($z \leq -0{,}1$ m), yang biasanya merupakan galat pengukuran.

## 7.2 Voxel Downsampling (Grade AB)

Fungsi `voxel_downsample` menerapkan Persamaan (3) dengan `np.floor` dan `np.unique`, lalu mengembalikan pusat tiap voxel yang terisi. Ukuran voxel default 0,15 m dan dapat diubah dengan `--voxel`. Skrip mencetak jumlah titik sebelum dan sesudah serta persentase reduksinya.

## 7.3 Pewarnaan Berdasarkan Ketinggian (Grade A)

Nilai $z$ tiap titik dinormalisasi lalu dipetakan ke *colormap* `RdYlGn_r`: hijau untuk titik rendah (tanah), kuning untuk menengah, dan merah untuk titik tertinggi. Dengan $r = z_{\max} - z_{\min}$ dan $\alpha = 0{,}25$ (`pad`), nilai ternormalisasinya adalah

$$\hat{z} = \frac{z - (z_{\min} - \alpha r)}{r\,(1 + \alpha)}. \tag{11}$$

Parameter $\alpha$ menggeser ujung bawah skala sehingga tanah tampil hijau muda, bukan hijau paling gelap.

## 7.4 Visualisasi dan Keluaran

- **Mode 3D:** voxel digambar sebagai penanda persegi berwarna pada plot 3D *matplotlib*, bersama jalur terbang drone (garis biru). Skala sumbu dibuat sesuai proporsi

metrik asli. Gambar disimpan sebagai `map3d.png`. File `cloud_colored.ply` (maksimal sekitar 200 000 titik) dan `voxels_colored.ply` juga disimpan sehingga dapat dibuka di perangkat lunak seperti MeshLab atau CloudCompare.

- **Mode 2D:** titik hasil pindai digambar sebagai peta tampak atas bersama jalur drone, disimpan sebagai `map2d.png`.
- **Opsi tambahan:** `--raw` menampilkan *point cloud* mentah, `--size` mengatur ukuran penanda voxel, dan `--o3d` membuka tampilan voxel interaktif dengan Open3D bila terpasang.

# 8 Cara Menjalankan

## 8.1 Persiapan

```
pip install coppeliasim-zmqremoteapi-client numpy matplotlib
pip install open3d # opsional
```

Selanjutnya:

1. Buka CoppeliaSim, lalu muat scene dengan menu *File* → *Open scene* dan pilih `ETS Mapping.ttt`.
2. Pastikan `mapping_sensor` sudah dibuat sesuai Tabel 3, menjadi *child* dari `Quadcopter`, dan opsi *Explicit handling* aktif.

## 8.2 Pengambilan Data

```
python drone_mapping.py --mode path # Grade C
python drone_mapping.py --mode 2d # Grade BC
python drone_mapping.py --mode 3d # Grade B
```

Simulasi tidak boleh dihentikan manual. Tunggu sampai terminal mencetak `[done]`.

## 8.3 Pengolahan Data

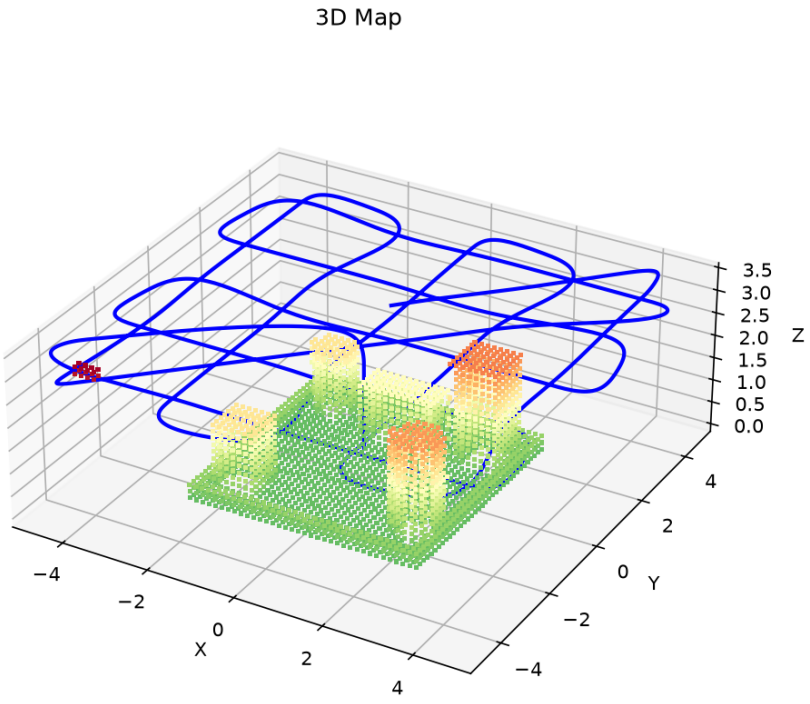
```
python postprocess.py --mode 3d --voxel 0.15 # Grade AB dan A
python postprocess.py --mode 2d # peta 2D
python postprocess.py --mode 3d --voxel 0.15 --o3d # tampilan Open3D
```

### 8.4 Pemecahan Masalah

- **Galat "object not tagged for explicit handling"**: opsi *Explicit handling* pada `mapping_sensor` belum dicentang.
- **Data kosong atau titik hilang**: pastikan pemanggilan `sim.getVisionSensorDepth(sensor, 0)` memakai argumen `0` (kedalaman ternormalisasi).
- **Peta terbalik**: ubah `FLIP_U` atau `FLIP_V` menjadi `True` lalu pindai ulang.
- **Quadcopter tidak ditemukan**: pastikan model *Quadcopter* berada di scene beserta *dummy* `target`.

## 9 Hasil dan Pembahasan

### 9.1 Hasil Visualisasi




Gambar 3: Peta 3D hasil voxel downsampling dengan pewarnaan berdasarkan ketinggian, beserta jalur terbang

## 9.2 Ringkasan Angka Hasil

Angka berikut dicetak oleh `postprocess.py --mode 3d --voxel 0.15` di terminal (baris `raw points`, `voxels`, dan `height range`). Salin nilainya ke tabel.

Tabel 5: Ringkasan hasil pemindaian 3D

| **Besaran** | **Nilai** |
|---|---|
| Jumlah titik mentah (setelah pembersihan) | ... |
| Ukuran voxel | 0,15 m |
| Jumlah voxel | ... |
| Reduksi jumlah titik | ... % |
| Rentang tinggi $z_{\min} - z_{\max}$ | ... – ... m |

## 9.3 Pembahasan

**Pemilihan sudut kemiringan** 45°**.** Dengan sensor menghadap lurus ke depan, hanya dinding vertikal yang terlihat. Dengan sensor menghadap lurus ke bawah, hanya permukaan atas dan tanah yang terlihat. Kemiringan 45° adalah kompromi yang memungkinkan tanah, dinding, dan bagian atas objek terekam dalam satu frame. Pada ketinggian 3 m, sinar teratas (sekitar 15° di bawah horizontal) masih mengenai tanah dalam jangkauan *far plane* 12 m.

**Dua tahap lintasan.** Pada satu arah lintasan, bagian belakang objek tertutup oleh objek itu sendiri (*occlusion*). Tahap kedua yang tegak lurus terhadap tahap pertama melihat objek dari arah lain sehingga cakupan permukaan lebih lengkap.

**Pengaruh ukuran voxel.** Voxel yang lebih besar menghasilkan jumlah titik lebih sedikit dan peta lebih ringan, tetapi detail bentuk objek berkurang. Voxel yang lebih kecil mempertahankan detail dengan biaya penyimpanan lebih besar. Nilai 0,15 m dipilih sebagai titik tengah untuk objek berukuran sekitar 1 m.

**Keterbatasan.**

- Posisi sensor diambil dari posisi drone di simulator, bukan dari estimasi sensor sehingga tidak ada galat lokalisasi. Pada drone nyata, galat posisi akan membuat peta menjadi kabur atau bergeser.
- Orientasi sensor dibuat langsung dari arah waypoint, bukan dibaca dari kemiringan aktual badan drone.
- Sensor simulasi tidak memiliki derau, sehingga permukaan tampak lebih bersih dari data sensor sebenarnya.

- Rute dibuat tetap dan tidak ada penghindaran rintangan. Aman karena ketinggian terbang lebih tinggi dari semua objek.
- Data baru tersimpan setelah seluruh lintasan selesai, sehingga penghentian simulasi di tengah jalan membuat data hilang.

# 10 Kesimpulan

Proyek ini berhasil membangun sistem pemetaan otonom pada simulator CoppeliaSim yang memenuhi seluruh tingkatan tugas. Drone dikendalikan lewat *dummy* `target` yang digerakkan skrip Python melalui ZMQ Remote API dengan mode *stepping* agar sinkron. Pemindaian 2D memakai satu baris piksel yang diputar seperti lidar, sedangkan pemindaian 3D memakai kamera kedalaman miring 45° dengan lintasan *lawn-mower* dua tahap. Setiap piksel kedalaman diproyeksikan balik ke koordinat dunia melalui model *pinhole* dan transformasi pose sensor. Pada tahap pengolahan, *point cloud* dibersihkan, direduksi dengan *voxel downsampling*, dan diwarnai berdasarkan ketinggian sehingga menghasilkan peta 3D yang ringan dan mudah dibaca.

Pengembangan selanjutnya dapat berupa penambahan derau sensor, estimasi pose dengan odometri atau SLAM, penghindaran rintangan, serta lintasan adaptif yang menyesuaikan diri dengan bentuk area.

# Lampiran: Tautan Proyek

Tautan berikut sama dengan yang tertera pada halaman sampul. Tempel kembali di sini bila diperlukan.

**Repositori GitHub**
https://github.com/dante011111/ETS-SRO-Drone-Mapping

**Video Demonstrasi 1**
https://drive.google.com/file/d/1lNExSDm8HoHWYaXFC_JeajzcxdrlOUzW/view?usp=drive_link
*Isi video: Video Proses Mapping.*

**Video Demonstrasi 2**
https://drive.google.com/file/d/11Uf6JHnGKmAgEEihSATajqXOj-cRtIVX/view?usp=drive_link
*Isi video: Video Postprocessing Plot Hasil Mapping.*